

สรุปเนื้อหาสอบ Midterm — Machine Learning

ครอบคลุม 6 หัวข้อ: Introduction to ML, Linear Regression, Generalization, Performance Estimation, Model Selection, Regularization

หัวข้อที่ 1: Introduction to Machine Learning

1.1 Machine Learning คืออะไร

ML คือการทำให้คอมพิวเตอร์ "เรียนรู้" จากข้อมูล แทนที่จะเขียนโค้ดสั่งทุกกฎเอง ตัวอย่างง่ายๆ คือ Teachable Machine ของ Google ที่มี 3 ขั้นตอน:

1. **Gather** — เก็บตัวอย่างข้อมูล (รูป/เสียง/ท่าทาง) จัดกลุ่มเป็น class
2. **Train** — ให้โมเดลเรียนรู้จากตัวอย่างเหล่านั้น
3. **Export** — นำโมเดลไปใช้งานจริง

1.2 ประเภทของปัญหา ML

Supervised Learning — มีทั้ง input (x) และ output/label (y) ที่ถูกต้อง โจทย์คือหา f ที่ทำให้ $y = f(x)$

- **Regression:** output เป็นตัวเลขต่อเนื่อง (Quantitative/Numeric) เช่น ทำนายความสูงจากความกว้าง
- **Classification:** output เป็นหมวดหมู่ (Qualitative/Nominal) เช่น จำแนกส้ม vs มะนาว

Unsupervised Learning — มีแค่ x ไม่มี label y โจทย์คือหาโครงสร้างในข้อมูลเอง

- **Clustering:** จัดกลุ่มข้อมูลที่คล้ายกันไว้ด้วยกัน (ไม่รู้ล่วงหน้าว่ามีกี่กลุ่มหรือกลุ่มไหนคืออะไร)

เกร็ดเสริม: ถ้ารู้ f แล้วอยากหา x ที่ทำให้ผลลัพธ์ตรงเงื่อนไข → เป็นปัญหา **Optimization** ไม่ใช่ ML โดยตรง

1.3 Analytical Purpose (มุมมองอีกแบบของ ML)

ประเภท	ความหมาย	ตัวอย่าง
Descriptive	อธิบายสภาพปัจจุบัน	"อ่านหนังสือมาก → คะแนนสูง"
Predictive	สร้างโมเดลทำนายอนาคต	$Score = 20 + 10 \times StudyHours - 5 \times ScreenTime$
Prescriptive	แนะนำ action ที่ควรทำ	"ถ้าเล่นมือถือ 2 ชม. ควรอ่านหนังสืออย่างน้อย 5 ชม. เพื่อให้คะแนนเกิน 60"

1.4 CRISP-DM (Cross Industry Standard Process for Data Mining)

กระบวนการทำงานมาตรฐาน 6 ขั้นตอน (ออกสอบเรื่องลำดับขั้นตอนบ่อย):

ขั้น	ชื่อ	ทำอะไร
1	เข้าใจปัญหา	ระบุ Business Objective, นิยามปัญหาเป็น Regression/Classification/Clustering
2	เข้าใจข้อมูล	ดูข้อมูลด้วย scatter plot, histogram, box plot, correlation
3	เตรียมข้อมูล	จัดการ missing value, outlier, สกัด feature ใหม่ (feature extraction)
4	สร้างแบบจำลอง	เลือกโมเดล, ปรับ parameter, ประเมินผล
5	ประเมินผล	เทียบกับ Business Success Criteria
6	นำไปใช้งาน	ใช้โมเดลทำนายข้อมูลใหม่จริง

1.5 Overfitting (เกริ่นนำ)

เกิดเมื่อโมเดลซับซ้อนเกินไป จนไปจับ "สัญญาณรบกวน" (noise) ในข้อมูล training แทนที่จะจับ pattern จริง ผลคือ **Training error** ต่ำมาก แต่ **Testing error** สูง

จาก Classifier Playground: โมเดลง่าย (Logistic Regression, Bayes, SVM เชิงเส้น) จับ pattern ทั่วไปได้ดีแต่พลาตถ้า boundary โค้ง ส่วนโมเดลยืดหยุ่นสูง (Tree, kNN) ปรับตามข้อมูลได้ดีแต่เสี่ยง overfit ถ้าไม่คุม hyperparameter

1.6 Is Learning Feasible?

เป้าหมายจริงของ ML คือให้โมเดลทำนายข้อมูลนอกชุดที่เคยเห็น (outside the data set) ได้ถูกต้อง — นี่คือการวัดของ **Generalization** (หัวข้อ 3) มีแนวคิดเรื่อง Interpolation (ทำนายในช่วงข้อมูล) vs Extrapolation (ทำนายนอกช่วงข้อมูล)

หัวข้อที่ 2: Linear Regression

ทุกโมเดล ML ประกอบด้วย 3 องค์ประกอบ: **Representation → Evaluation → Optimization**

2.1 Covariance และ Correlation

Covariance: บอกว่า x กับ y "ไปด้วยกัน" แค่ไหน

$$cov(x, y) = \frac{1}{n} \sum_i (x_i - \bar{x})(y_i - \bar{y})$$

Correlation (r): covariance ที่ทำให้เป็นมาตรฐาน (ไม่ขึ้นกับหน่วย) อยู่ในช่วง -1 ถึง 1 ยิ่งใกล้ ±1 ยิ่งเกาะเส้นตรงชัด

ข้อควรระวัง: Correlation ≠ Regression — correlation บอกแค่ความสัมพันธ์ ส่วน regression คือการสร้างสมการทำนายค่าจริง

2.2 Representation

Simple Linear Regression (ตัวแปรเดียว): $h(x) = w_0 + w_1x$

สัญลักษณ์สำคัญ:

- m = จำนวนข้อมูล (จำนวนแถว), n = จำนวนตัวแปร (features)

- $x^{(i)}$ = ตัวอย่างที่ i , $x_j^{(i)}$ = feature ที่ j ของตัวอย่างที่ i

Multiple Linear Regression (หลายตัวแปร): $h(x) = w_0 + w_1x_1 + w_2x_2 + \dots + w_dx_d$

เขียนแบบเมทริกซ์: $y = Xw + \varepsilon$ โดย X ขนาด $m \times n$

2.3 Evaluation

SSE (Sum of Squared Errors):

$$SSE = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

แนวคิด: แต่ละจุดที่เส้นทำนายผิดพลาดสร้าง residual $e_i = y_i - \hat{y}_i$ ยกกำลังสอง (เพื่อไม่ให้ error บวก-ลบหักล้างกัน และลงโทษ error ใหญ่มากขึ้น) — ยิ่ง SSE น้อย เส้นยิ่งพิดดี

R² (Coefficient of Determination):

$$R^2 = 1 - \frac{SSE}{SST}$$

- R^2 เข้าใกล้ 1 $\rightarrow$ เส้นอธิบายข้อมูลได้ดีมาก
- R^2 เข้าใกล้ 0 $\rightarrow$ เส้นแทบไม่ดีกว่าการทายด้วยค่าเฉลี่ย $\bar{y}$
- กรณีตัวแปรเดียว: $R^2 = r^2$ (เป็นกรณีพิเศษของ correlation)

Metric อื่นๆ:

Metric	สูตร	ความหมาย	ทิศทางที่ดี
MAE	$\frac{1}{n} \sum \ y_i - \hat{y}_i\ $	ความผิดพลาดเฉลี่ย	ต่ำ
RMSE	$\sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2}$	ลงโทษ outlier แรงกว่า, หน่วยเดียวกับ y	ต่ำ
R ²	$1 - \frac{SSE}{SST}$	สัดส่วนที่โมเดลอธิบายได้	สูง

2.4 Optimization

Cost Function (MSE): $E(w) = \frac{1}{2n} \sum_{i=1}^n (h(x^{(i)}) - y^{(i)})^2$

วิธีที่ 1: Normal Equation (แก้สมการทางตรง)

ตั้งอนุพันธ์ = 0 แล้วแก้สมการได้คำตอบแบบ closed-form ในขั้นตอนเดียว

- กรณีตัวแปรเดียว: $\hat{w}_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$, $\hat{w}_0 = \bar{y} - \hat{w}_1\bar{x}$
- กรณีหลายตัวแปร (รูปเมทริกซ์): $w = (X^T X)^{-1} X^T y$

วิธีที่ 2: Gradient Descent (ไล่ลงทีละก้าว)

สุ่มค่า w เริ่มต้น แล้วปรับในทิศทางข้ามกับ gradient จนถึงจุดต่ำสุด:

$$\frac{\partial E}{\partial w_0} = \frac{1}{n} \sum (h(x^{(i)}) - y^{(i)}), \quad \frac{\partial E}{\partial w_1} = \frac{1}{n} \sum (h(x^{(i)}) - y^{(i)}) x^{(i)}$$

Learning rate ควบคุมขนาดก้าว: สูงเกินไป → แกว่งไม่ลู่เข้า (diverge), ต่ำเกินไป → ลู่เข้าช้า

เปรียบเทียบ: Normal Equation เหมาะกับข้อมูลไม่ใหญ่มาก (ต้องกลับเมทริกซ์ $X^T X$ ซึ่งหนักเมื่อ n ใหญ่) ส่วน Gradient Descent เหมาะกับข้อมูลขนาดใหญ่

2.5 ปัญหา Invertible Matrix (Normal Equation ใช้ไม่ได้เมื่อใด)

$X^T X$ กลับเมทริกซ์ไม่ได้เมื่อ:

- **Multicollinearity** — ตัวแปรซ้ำซ้อนกัน (เช่น ขนาดบ้านหน่วยตารางฟุต กับ ตารางเมตร)
- **$d > n$** — จำนวนตัวแปรมากกว่าจำนวนตัวอย่าง

วิธีแก้: ตัดตัวแปรที่ซ้ำซ้อนออก หรือใช้ **Regularization** (เชื่อมโยงหัวข้อ 6)

หัวข้อที่ 3: Generalization

3.1 Ein vs Eout

- **Ein (In-sample Error)** = error จากข้อมูลที่ใช้เทรน
- **Eout (Out-of-sample Error)** = error จากข้อมูลที่ไม่เคยเห็น

เป้าหมายจริงของ ML คือทำให้ Eout ต่ำ ไม่ใช่ Ein เพราะโมเดลต้องใช้กับข้อมูลใหม่จริง

จาก Learning Curve: เมื่อ n เพิ่มขึ้น — Ein มีแนวโน้มเพิ่มขึ้นเล็กน้อย, Eout มีแนวโน้มลดลง, ทั้งสองเส้นลู่เข้าหากัน

3.2 Bias-Variance Decomposition (แก่นของหัวข้อนี้)

$$E_D[E_{out}] = \sigma^2 + \text{bias}^2 + \text{variance}$$

องค์ประกอบ	ความหมาย	ลดได้ด้วย
bias ²	แบบจำลองเฉลี่ยห่างจากเป้าหมายจริง (เรียบง่ายเกินไป)	เพิ่มความซับซ้อนของ โมเดล (M สูงขึ้น)
variance	แบบจำลองแกว่งตามชุดข้อมูล (ยืดหยุ่นเกินไป)	เพิ่มข้อมูล (n) หรือลดความซับซ้อน
σ^2 (noise)	ความรบกวนในข้อมูลเอง	ลดไม่ได้ (irreducible)

จำง่าย ๆ:

- **Bias สูง** = เรียบเกินไป (underfitting) — โมเดลง่ายเกินไปจนจับ pattern จริงไม่ได้
- **Variance สูง** = ยืดหยุ่นเกินไป (overfitting) — โมเดลไปจับ noise ของแต่ละชุดข้อมูล
- Ein ต่ำ แต่ Eout สูงมาก → เชื่อมโยงกับ variance สูง (overfit)

3.3 ตัวอย่างจริง (โจทย sin function, ทายด้วยข้อมูล 2 จุด)

Model	bias ²	variance	Eout
ค่าคงตัว (constant)	0.50	0.25	0.75
เชิงเส้น (linear)	0.21	1.69	1.90

ผลลัพธ์ที่น่าประหลาดใจ: เมื่อข้อมูลน้อยมาก โมเดลง่ายกว่า (constant) ชนะโมเดลเชิงเส้น เพราะแม้ bias สูงกว่าแต่ variance ต่ำกว่ามาก → โมเดลซับซ้อนกว่าไม่ได้แปลว่าดีกว่าเสมอไป โดยเฉพาะเมื่อข้อมูลมีน้อย

3.4 ความสัมพันธ์กับความซับซ้อนของโมเดล (Tradeoff)

ความซับซ้อนโมเดล น้อย —————> มาก
bias²: สูง —————> ต่ำ
variance: ต่ำ —————> สูง

- โมเดลง่าย → bias สูง, variance ต่ำ → **underfit**
- โมเดลซับซ้อน → bias ต่ำ, variance สูง → **overfit**
- จุดที่ดีที่สุด = ตรงกลางที่ (bias² + variance) ต่ำที่สุด

3.5 ผลของจำนวนข้อมูล (n)

- เพิ่มข้อมูล → ลด variance ได้ แต่ไม่ช่วยลด bias
- เมื่อ $n \rightarrow \infty$: variance $\rightarrow 0$ ทำให้ $E_{out} = E_{in} = bias$

สรุปวิธีลด Eout: (1) เพิ่มความซับซ้อนของโมเดล → ลด bias หรือ (2) เพิ่มจำนวนข้อมูล → ลด variance

หัวข้อที่ 4: Performance Estimation

4.1 Accuracy vs Precision

- Accuracy (ความแม่นยำ) = ความเอนเอียงต่ำ (Low Bias)
- Precision (ความเที่ยงตรง) = ความแปรปรวนต่ำ (Low Variance)

4.2 วิธีประเมินโมเดล 3 แบบ

1) Resubstitution: ใช้ข้อมูลชุดเดียวกันทั้ง train และ test → error ต่ำเกินจริง (optimistic) เพราะโมเดลเคย "เห็น" ข้อมูลทดสอบแล้ว — วัด Ein ไม่ใช่ Eout ไม่ควรใช้ตัดสินใจจริง

2) Holdout (Percentage Split): แบ่งข้อมูลเป็น Train/Test (เช่น 70/30)

- Train มากเกินไป (เช่น 90%) → Test set เล็ก → RMSE เฉลี่ยและ SD สูง (วัดผลไม่เสถียร)
- Train น้อยเกินไป (เช่น 10%) → โมเดลเรียนรู้ไม่พอ → RMSE เฉลี่ยและ SD สูงเช่นกัน
- ต้องหาสัดส่วนสมดุล (ทางปฏิบัตินิยม 70/30 หรือ 80/20)

3) Cross-Validation (k-fold): แบ่งข้อมูลเป็น k ส่วน วนทดสอบ k รอบ (แต่ละรอบ ใช้ 1 fold เป็น test, ที่เหลือเป็น train) แล้วเฉลี่ยผล

- k มาก → ข้อมูล train ในแต่ละรอบเยอะ → แม่นยำ (bias ต่ำ) แต่ผลอาจกว้าง
- k น้อย → ข้อมูล train น้อยลง → แม่นยำน้อยกว่า
- คำนิยมใช้ในทางปฏิบัติ: **k = 10** (สมดุลระหว่าง bias/variance ของการประเมิน)

4.3 Metrics

Metric	สูตร	ความหมาย
R ²	$1 - \frac{SSE}{SST}$	สัดส่วนความผันแปรที่อธิบายได้ (ยิ่งสูงยิ่งดี)
MAE	$\frac{1}{n} \sum \ y_i - \hat{y}_i\ $	ค่าเฉลี่ย error สัมบูรณ์
RMSE	$\sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2}$	ลงโทษ error ใหญ่มากกว่า MAE
RAE	เทียบ MAE โมเดล กับ MAE การทายด้วยค่าเฉลี่ย (%)	ดีกว่าเดาค่าเฉลี่ยแค่ไหน
RRSE	เทียบ RMSE โมเดล กับ RMSE การทายด้วยค่าเฉลี่ย (%)	เช่นเดียวกับ RAE แต่ใช้ RMSE

ตัวอย่างผลจริงจากการทดลอง:

Metric	Training (Resubstitution)	Holdout	Cross-Validation
Correlation coefficient	0.93	0.92	0.89
MAE	6.20	7.96	7.31
RMSE	6.70	8.48	8.10
RAE	43.55%	50.56%	44.54%
RRSE	37.86%	40.77%	39.37%

Resubstitution ให้ error ต่ำที่สุดเสมอ (ลำเอียง มอง โลก ในแง่ดีเกินไป) — Holdout กับ CV ใกล้เคียงความจริงมากกว่า

4.4 สรุปเปรียบเทียบ

วิธี	Accuracy (bias)	Precision (variance)	หมายเหตุ
Resubstitution	ดูดีเกินจริง	—	ไม่ควรใช้ประเมินจริง
Holdout	ขึ้นกับสัดส่วน split	ต่ำกว่า CV (ผลกว้างง่าย)	เร็ว ทำง่าย
Cross-Validation	ดี	สูงกว่า (เสถียรกว่า)	ใช้เวลาคำนวณมากกว่า

หัวข้อที่ 5: Model Selection

5.1 Train / Validation / Test

ชุดข้อมูล	จุดประสงค์	ตัวอย่าง
Training	หา parameter ของ โมเดล	$w_0, w_1, \dots$
Validation	หา hyperparameter ที่ดีที่สุด	degree ของ polynomial, ค่า λ
Testing	ประเมินผลสุดท้าย (reference error)	วัดผลจริงกับข้อมูลที่ไม่เคยใช้เลย

สำคัญมาก: ห้ามใช้ test set มาช่วยเลือก hyperparameter — ถ้าใช้จะทำให้ test error สุดท้าย มองโลกในแง่ดีเกินจริง (ไม่ใช่ Eout จริง) ต้องมี validation set/cross-validation แยกต่างหาก แล้วเก็บ test set ไว้วัดผลครั้งสุดท้าย

5.2 ทำไม RSS/ R^2 ใช้เลือก โมเดลไม่ได้โดยตรง

โมเดลที่มีตัวแปรมากที่สุดจะมี RSS ต่ำสุด และ R^2 สูงสุดเสมอ (คำนวณจาก training error) — แต่เป้าหมายจริงคือ test error ต่ำ ดังนั้น RSS/ R^2 ไม่เหมาะเลือก โมเดลที่มีจำนวนตัวแปรต่างกัน ต้องใช้ validation/CV error แทน

5.3 การทดลอง Polynomial Degree (ตัวอย่างคลาสสิก)

ทดลองสร้าง Polynomial Regression ทาย sin function เปลี่ยน degree (1, 3, 8) และจำนวนข้อมูล (10, 20, 40, 80) เทียบ Training error กับ CV error

- Degree ต่ำ (=1): training error และ CV error สูงทั้งคู่ → **underfit**, bias สูง
- Degree สูง (=8): training error ต่ำมาก แต่ CV error สูงกว่ามาก โดยเฉพาะเมื่อข้อมูลน้อย → **overfit**, variance สูง
- เพิ่มจำนวนข้อมูล → ช่องว่างระหว่าง training error กับ CV error ของ โมเดล degree สูง แคบลง
- Noise ในข้อมูลทำให้ overfitting รุนแรงขึ้น โดยเฉพาะ degree สูง + ข้อมูลน้อย

5.4 Subset Selection — เลือกตัวแปรที่เหมาะสม

Best Subset Selection: ลองทุกชุดค่าผสมที่เป็นไปได้ (2^p โมเดล จาก p ตัวแปร) เลือกตัวที่ดีที่สุด ในแต่ละขนาด แล้วใช้ validation/CV เลือกขนาดที่ดีที่สุด

- ข้อดี: รับประกัน โมเดลที่ดีที่สุด ในแต่ละขนาด
- ข้อเสีย: คำนวณหนักมากเมื่อ p ใหญ่ (2^p โตเร็ว) และพื้นที่ค้นหาใหญ่อาจนำไปสู่การ overfit บน training data (variance ของสัมประสิทธิ์สูง)

Forward Stepwise Selection: เริ่มจากไม่มีตัวแปรเลย เพิ่มทีละตัว (เลือกตัวที่ปรับปรุง โมเดลได้มากที่สุด ในแต่ละขั้น) จนครบทุกตัวแปร

- ข้อดี: คำนวณเร็วกว่า best subset มาก
- ข้อเสีย: ไม่รับประกันว่าได้โมเดลที่ดีที่สุดจากทุก 2^p โมเดล (เป็นการค้นหาแบบ greedy)

Backward Stepwise Selection: เริ่มจาก โมเดลที่มีตัวแปรครบทุกตัว ตัดทีละตัวที่มีประโยชน์น้อยที่สุด ออก จนเหลือ โมเดลว่าง

- ข้อจำกัด: ต้องมี $n > p$ (จำนวนข้อมูลมากกว่าจำนวนตัวแปร) เพื่อสร้าง โมเดลเต็มรูปแบบได้ตั้งแต่ต้น

5.5 การเลือกโมเดลสุดท้าย

ใช้ **Cross-Validation** ประเมิน error ของแต่ละโมเดลผู้เข้าแข่งขัน แล้วเลือกตัวที่มี CV error ต่ำสุด — ห้ามใช้ training error/RSS/ R^2 เพราะจะเอนเอียงไปทาง โมเดลซับซ้อนที่สุดเสมอ

หัวข้อที่ 6: Regularization

6.1 แนวคิดหลัก

3 วิธีจัดการ โมเดลซับซ้อน/ตัวแปรเยอะ:

1. **Subset Selection** — เลือกตัวแปรบางส่วน (discrete process)
2. **Shrinkage (Regularization)** — ใช้ทุกตัวแปร แต่บีบค่าสัมประสิทธิ์เข้าใกล้ศูนย์ (continuous process)
3. **Dimension Reduction** — ฉายตัวแปร p ลง subspace มิติต่ำกว่า ($M < p$)

6.2 Ridge Regression (L2 Regularization)

Cost Function:

$$E(w) = \frac{1}{2n} \sum_{i=1}^n (h(x^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2n} \sum_{j=1}^d w_j^2$$

λ (regularization parameter): $\lambda=0 \rightarrow$ เหมือน Linear Regression ปกติ, λ มาก $\rightarrow$ บีบ w เข้าใกล้ 0 มากขึ้น, $\lambda \rightarrow \infty \rightarrow w$ ทุกตัวเข้าใกล้ 0

Gradient:

$$\frac{\partial E}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (h(x^{(i)}) - y^{(i)}) x_j^{(i)} \quad (j = 0, \text{ ไม่ถูกลงโทษ})$$

$$\frac{\partial E}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (h(x^{(i)}) - y^{(i)}) x_j^{(i)} + \lambda w_j \quad (j \geq 1)$$

รูปเมทริกซ์ (Normal Equation ของ Ridge):

$$w = (X^T X + \lambda I')^{-1} X^T y$$

โดย I' คือเมทริกซ์เอกลักษณ์ที่ตำแหน่ง w_0 เป็น 0 (ไม่ลงโทษ intercept) — สังเกตว่ารูปแบบนี้ช่วยแก้ปัญหา matrix ไม่ invertible จากหัวข้อ 2 ได้ด้วย เพราะการบวก $\lambda I'$ ทำให้เมทริกซ์กลับด้านได้เสมอ แม้ตอน $d > n$

คุณสมบัติ: บีบค่า w ให้เล็กลงเรื่อยๆ แต่ไม่เป็นศูนย์เป๊ะ $\rightarrow$ เก็บตัวแปรไว้ครบทุกตัวเสมอ เพียงลดอิทธิพลลง

6.3 Lasso Regression (L1 Regularization)

Cost Function:

$$E(w) = \frac{1}{2n} \sum_{i=1}^n (h(x^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2n} \sum_{j=1}^d |w_j|$$

ต่างจาก Ridge ตรงที่ penalty ใช้ค่าสัมบูรณ์ (L1 norm) แทนกำลังสอง

Variable Selection Property: Lasso บีบค่า w ให้เป็นศูนย์เป๊ะได้จริงเมื่อ λ มากพอ $\rightarrow$ ตัวแปรที่ไม่สำคัญถูกตัดออกจากโมเดลไปเลย $\rightarrow$ Lasso ทำหน้าที่ **feature selection** ในตัว ต่างจาก Ridge ที่เก็บทุกตัวแปรไว้เสมอ

6.4 ทำไม Lasso ทำให้ w เป็นศูนย์ได้ แต่ Ridge ไม่ได้ (สัญชาตญาณเชิงเรขาคณิต)

Ridge จำกัด w ให้อยู่ในวงกลม (L2 ball) ส่วน Lasso จำกัด w ให้อยู่ในรูปเพชร/สี่เหลี่ยมข้าวหลามตัด (L1 ball, มีมุมแหลม)

- จุดสัมผัสของ contour error กับวงกลมมีโอกาสตกที่ตำแหน่งใดก็ได้บนขอบ (w ไม่ค่อยเป็น 0)
- จุดสัมผัสกับรูปเพชรมีแนวโน้มตกที่มุมแหลม ซึ่งบางแกน (บาง w) มีค่าเท่ากับ 0 พอดี

6.5 Bias-Variance Tradeoff ของ Ridge/Lasso

λ	Bias	Variance	สถานะ
$\lambda = 0$	ต่ำ	สูง	เหมือน Linear Regression ปกติ เสี่ยง overfit
λ เพิ่มขึ้น	เพิ่มขึ้น	ลดลง	โมเดลเรียบง่ายขึ้นเรื่อยๆ
$\lambda \rightarrow \infty$	สูงมาก	ต่ำมาก	underfit (โมเดลเกือบคงที่)

การเลือก λ ที่ดีที่สุด: ใช้ **Cross-Validation** — ลอง λ หลายค่า วัด CV error แล้วเลือก λ ที่ CV error ต่ำสุด (λ คือ hyperparameter ต้องเลือกด้วย validation set ไม่ใช่ training set — สอดคล้องกับหัวข้อ 5)

6.6 เปรียบเทียบสรุป Ridge vs Lasso

ประเด็น	Ridge (L2)	Lasso (L1)
Penalty	$\sum w_j^2$	$\sum \ w_j\ $
ค่า w เป็น 0 ได้ไหม	ไม่ได้ (เข้าใกล้ 0 เท่านั้น)	ได้ (ตัดตัวแปรทิ้งจริง)
Feature selection	ไม่มีในตัว	มีในตัว
เหมาะกับ	ตัวแปรทุกตัวมีประโยชน์บ้าง	มีตัวแปรจำนวนมาก คาดว่ามีแค่บางตัวสำคัญจริง (sparse)

ภาพรวมความเชื่อมโยงทั้ง 6 หัวข้อ

1. Intro to ML → รู้จักประเภทปัญหา (Regression/Classification/Clustering), CRISP-DM
2. Linear Regression → สร้างโมเดล (Representation, Evaluation, Optimization)
3. Generalization → ทำไม overfit เกิดขึ้น (Bias-Variance Decomposition)
4. Performance Estimation → วัด Error จริงยังไง (Resubstitution/Holdout/CV, Metrics)
5. Model Selection → เลือกโมเดล/hyperparameter ยังไง (Train/Validation/Test, Subset Selection)
6. Regularization → เทคนิคควบคุม bias-variance โดยตรง (Ridge/Lasso)

เช็กลิสต์ก่อนสอบ

- ☐ แยก Regression / Classification / Clustering ได้
- ☐ จำลำดับ CRISP-DM 6 ขั้นตอนได้
- ☐ เขียนสมการ Linear Regression, คำนวณ SSE/R²/MAE/RMSE ได้
- ☐ อธิบาย Normal Equation vs Gradient Descent และข้อดี-ข้อเสียได้
- ☐ อธิบาย Bias-Variance Decomposition และ tradeoff ได้
- ☐ เปรียบเทียบ Resubstitution / Holdout / Cross-Validation ได้
- ☐ อธิบาย Train/Validation/Test แยกกันทำไม
- ☐ อธิบาย Best Subset / Forward / Backward Selection ต่างกันอย่างไร
- ☐ เขียน Cost Function ของ Ridge และ Lasso ได้ อธิบายผลของ λ ได้
- ☐ อธิบายว่าทำไม Lasso ทำ feature selection ได้แต่ Ridge ไม่ได้